



AI ENGINEERING STACK

# Build a Production RAG Pipeline with LangChain

Step-by-step from raw documents to intelligent answers

Swipe to build it yourself →

---

AI Coach John

Follow

PROITBRIDGE

↻ Repost

# What is RAG?

**Retrieval-Augmented Generation.** Instead of relying on an LLM's training data alone, you fetch relevant documents at runtime and feed them as context. The LLM answers from your data, not from memory.

## ★ WHY IT MATTERS

- ✓ No hallucinations from stale training
- ✓ Works on private/custom data
- ✓ Always up to date

### Pipeline preview:

Document → Chunk → Embed → Store  
Query → Retrieve → Validate → Generate

# Install Dependencies

Get everything you need in two commands. We use HuggingFace for free embeddings, FAISS for vector storage, and Groq for a fast free LLM.

```
pip install langchain langchain_community  
pip install langchain_huggingface faiss-cpu  
pip install langchain_groq pypdf presidio-analyzer  
pip install presidio-anonymizer rank_bm25
```

## STEP 2

# Load Your Documents

Document loaders convert files into LangChain `Document` objects with `page_content` and `metadata`. Pick the loader that matches your source.

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("your_file.pdf")
documents = loader.load()

# Each document has:
# doc.page_content → the text
# doc.metadata      → {"source": "file.pdf", "page": 0}
print(f"Loaded {len(documents)} pages")
```



**Other loaders:** `TextLoader`, `WebBaseLoader`, `CSVLoader`, `WikipediaLoader`

# Preprocess & Clean

LangChain has no built-in cleaner - write your own. Remove noise like page numbers, headers, and extra whitespace before chunking.

**Garbage in = garbage retrieved.**

```
import re

def clean(text):
    text = re.sub(r'\s+', ' ', text)      # extra whitespace
    text = re.sub(r'Page \d+', '', text)  # page numbers
    text = re.sub(r'^\w\s\.,\!\?]', '', text)
    return text.strip()

for doc in documents:
    doc.page_content = clean(doc.page_content)
```

# Validate & Mask PII

Before anything touches the pipeline, sanitize the user query. Block prompt injections and mask personal data (emails, phones, IDs) to stay GDPR/HIPAA compliant.

```
from presidio_analyzer import AnalyzerEngine
from presidio_anonymizer import AnonymizerEngine

analyzer = AnalyzerEngine()
anonymizer = AnonymizerEngine()

def mask_pii(text):
    results = analyzer.analyze(text=text, language="en")
    return anonymizer.anonymize(text=text,
                                analyzer_results=results).text

def validate_query(query):
    banned = ["ignore previous", "jailbreak", "disregard"]
    for phrase in banned:
        if phrase in query.lower():
            raise ValueError("Invalid query detected")
    return mask_pii(query.strip())

safe_query = validate_query(user_query)
```

## ✂ STEP 5

# Chunk the Documents

Split cleaned documents into smaller overlapping pieces. Chunk size controls precision; overlap preserves context across boundaries. Start at 300 chars and tune from there.

```
from langchain.text_splitter import
RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=300,
    chunk_overlap=50,
    separators=["\n\n", "\n", ".", " ", " "]
)

chunks = splitter.split_documents(documents)
print(f"{len(chunks)} chunks created")
```

💡 **Rule of thumb:** `chunk_overlap` = 15–20% of `chunk_size`

## Add Metadata to Chunks

Tag every chunk with structured metadata - source, category, date, author. This powers filtered retrieval later so you only search relevant subsets.

```
for i, chunk in enumerate(chunks):
    chunk.metadata.update({
        "chunk_id": i,
        "category": "langchain",    # tag manually or extract
        "year": 2024,
        "source": "rag_guide.pdf"
    })

# Now each chunk carries:
# {"source": "rag_guide.pdf", "category": "langchain",
#  "year": 2024, "page": 0, "chunk_id": 42}
```



## STEP 7

# Create Embeddings

Convert each chunk into a numerical vector that captures its meaning. Similar chunks end up close together in vector space - that's what enables semantic search.

```
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-mpnet-base-v2",
    model_kwargs={"device": "cpu"}
)

# Test it
vector = embeddings.embed_query("What is RAG?")
print(f"Vector size: {len(vector)}") # 768 dimensions
```



Free alternatives: [all-MiniLM-L6-v2](#), [text-embedding-ada-002](#)

## Store in Vector DB

Index all chunk embeddings into FAISS for fast similarity search. In production, swap FAISS for Chroma or Qdrant which support richer metadata filtering.

```
from langchain_community.vectorstores import FAISS

# Build the index from chunks + embeddings
vectorstore = FAISS.from_documents(chunks, embeddings)

# Save to disk so you don't rebuild every run
vectorstore.save_local("faiss_index")

# Load later
vectorstore = FAISS.load_local("faiss_index", embeddings,
                               allow_dangerous_deserialization=True)
```

# Hybrid Search (BM25 + Dense)

Pure vector search misses exact keyword matches. BM25 catches those. Combine both with an ensemble retriever - you get the best of semantic meaning AND keyword precision.

```
from langchain_community.retrievers import BM25Retriever
from langchain.retrievers import EnsembleRetriever

bm25 = BM25Retriever.from_documents(chunks)
bm25.k = 5

dense = vectorstore.as_retriever(search_kwargs={"k": 5})

retriever = EnsembleRetriever(
    retrievers=[bm25, dense],
    weights=[0.4, 0.6] # tune based on your data
)
```



**Tip: increase BM25 weight for technical/exact queries, dense for conceptual ones.**

# Metadata Filtering

Narrow retrieval to a specific subset before similarity search runs. This is especially powerful for multi-tenant apps or domain-separated corpora.

```
# Use Chroma for production metadata filtering
from langchain_community.vectorstores import Chroma

vectorstore = Chroma.from_documents(chunks, embeddings)

filtered_retriever = vectorstore.as_retriever(
    search_kwargs={
        "k": 5,
        "filter": {
            "$and": [
                {"category": {"$eq": "langchain"}},
                {"year": {"$gte": 2023}}
            ]
        }
    }
)
```

</> Operators: `$eq`, `$ne`, `$gt`, `$gte`, `$lt`, `$lte`, `$in`, `$and`, `$or`

# Multi-Query Retrieval

A single query often misses relevant chunks phrased differently. Generate 3–5 rephrasings automatically, retrieve for each, then deduplicate – recall goes up significantly.

```
from langchain.retrievers.multi_query import MultiQueryRetriever

multi_retriever = MultiQueryRetriever.from_llm(
    retriever=retriever,
    llm=llm
)

# Query: "How does RAG work?"
# Auto-generates:
# → "Explain the RAG architecture"
# → "What is retrieval augmented generation?"
# → "Steps in a RAG pipeline"
# Retrieves all, deduplicates, returns combined set

docs = multi_retriever.invoke(safe_query)
```

# Contextual Compression

Retrieved chunks often contain irrelevant sentences alongside the useful ones. Compression strips those out - only the sentences that answer the query pass through to the LLM.

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import
EmbeddingsFilter

# No LLM call needed - uses embedding similarity
compressor = EmbeddingsFilter(
    embeddings=embeddings,
    similarity_threshold=0.76
)

compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=retriever
)

compressed_docs = compression_retriever.invoke(safe_query)
```

 **Benefit: fewer tokens sent to LLM → lower cost + sharper context.**

# Set Up the LLM

Initialize your LLM with temperature=0 for deterministic, factual answers. Groq gives you fast free inference on open models - great for prototyping.

```
from langchain_groq import ChatGroq

llm = ChatGroq(
    model_name="gemma2-9b-it",
    temperature=0,          # 0 = factual, 1 = creative
    api_key="your_groq_api_key"
)

# Test
response = llm.invoke("What is LangChain?")
print(response.content)
```



Other options:

[ChatOpenAI](#)

, [ChatAnthropic](#)

, [ChatOllama](#)

## Generate the Answer

Wire the retriever and LLM together with RetrievalQA. The chain fetches relevant chunks, injects them into the prompt, and the LLM answers from that context only.

```
from langchain.chains import RetrievalQA

rag_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",          # all chunks in one prompt
    retriever=compression_retriever,
    return_source_documents=True
)

result = rag_chain.invoke({"query": safe_query})

answer = result["result"]
sources = result["source_documents"]
```

 Chain types: [stuff](#) (fast), [map\\_reduce](#) (long docs), [refine](#)



# Validate the Output

Don't return LLM responses blindly. Check faithfulness against retrieved context and strip any PII that might have leaked through from source documents.

```
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

check_prompt = PromptTemplate.from_template("""
Context: {context}
Answer: {answer}
Is this answer supported by the context?
Reply ONLY: SUPPORTED or NOT_SUPPORTED
""")

checker = check_prompt | llm | StrOutputParser()

def validate_output(answer, docs):
    context = " ".join([d.page_content for d in docs])
    verdict = checker.invoke({"context": context, "answer": answer})
    if "NOT_SUPPORTED" in verdict:
        return "Could not find a reliable answer in the documents."
    return mask_pii(answer) # mask any leaked PII

final_answer = validate_output(answer, sources)
```

# Full Pipeline at a Glance

Every card in this carousel is one step. Put them together and you have a production-ready RAG system.

1. Install dependencies
2. Load documents
3. Preprocess & clean
4. Validate input + mask PII ← security
5. Chunk documents
6. Tag chunks with metadata
7. Create embeddings
8. Store in vector DB
9. Hybrid search (BM25 + dense) ← recall
10. Metadata filtering ← precision
11. Multi-query retrieval ← coverage
12. Contextual compression ← efficiency
13. Set up LLM
14. Generate answer
15. Validate output + mask PII ← trust



**Save this. Share it. Build with it.**

Follow for more AI engineering content